

1. A) Create the following relation for the student:

Student (regno:string,name: string, class: string, bdate: date, marks1:int,
marks1:int, marks2:int, marks3:int)

- i. Enter at least five tuples of the above relation
- ii. Demonstrate the usage of following clauses for the above relation
 - a. Where
 - b. Order By
 - c. Having
 - d. Group By
- iii. Demonstrate the usage of following clauses for the above relation
 - a. Sum
 - b. Avg
 - c. Count
 - d. Like
 - e. Between
 - f. Max & Min
- iv. Demonstrate the rollback and commit command for the above relation

-- CREATE TABLE

```
CREATE TABLE Student (  
    regno VARCHAR(10),  
    name VARCHAR(50),  
    class VARCHAR(10),  
    bdate DATE,  
    marks1 INT,  
    marks2 INT,  
    marks3 INT  
);
```

-- INSERT AT LEAST FIVE TUPLES

```
INSERT INTO Student VALUES  
( 'S001', 'Abdul Sattar', 'BCA I', '2007-12-05', 85, 78, 92),  
( 'S002', 'Shanif Ahmed', 'BCA VI', '2005-09-07', 90, 88, 95),  
( 'S003', 'Ruvid', 'BCA VI', '2005-03-22', 70, 65, 80),  
( 'S004', 'Suraiya', 'BCA IV', '2005-02-11', 60, 75, 68),  
( 'S005', 'Kurath', 'BCA I', '2005-01-15', 95, 89, 97);
```

-- DISPLAY ALL RECORDS

```
SELECT * FROM Student;
```

-- II. Demonstrate usage of clauses

-- a) WHERE

-- Display records where marks1 > 80

```
SELECT *  
FROM Student  
WHERE marks1 > 80;
```

-- b) ORDER BY

-- Sort records in descending order of marks2

```
SELECT *  
FROM Student  
ORDER BY marks2 DESC;
```

-- c) GROUP BY

-- Average of marks1 grouped by class

```
SELECT class, AVG(marks1) AS avg_marks1  
FROM Student  
GROUP BY class;
```

-- d) HAVING

-- Display classes having average marks1 > 75

```
SELECT class, AVG(marks1) AS avg_marks1  
FROM Student  
GROUP BY class  
HAVING AVG(marks1) > 75;
```

-- III. Demonstrate aggregate and other clauses

-- a) SUM

-- Total of marks1

```
SELECT SUM(marks1) AS total_marks1  
FROM Student;
```

-- b) AVG
-- Average of marks2

```
SELECT AVG(marks2) AS avg_marks2  
FROM Student;
```

-- c) COUNT
-- Total number of students

```
SELECT COUNT(*) AS total_students  
FROM Student;
```

-- d) LIKE
-- Display names ending with 'i'

```
SELECT *  
FROM Student  
WHERE name LIKE '%i';
```

-- e) BETWEEN
-- Display students whose marks3 is between 70 and 90

```
SELECT *  
FROM Student  
WHERE marks3 BETWEEN 70 AND 90;
```

-- f) MAX and MIN
-- Highest and lowest marks3

```
SELECT  
    MAX(marks3) AS highest_marks3,  
    MIN(marks3) AS lowest_marks3  
FROM Student;
```

-- IV. Demonstrate COMMIT and ROLLBACK

START TRANSACTION;

UPDATE Student
SET marks1 = 100
WHERE regno = 'S003';

ROLLBACK;

COMMIT;

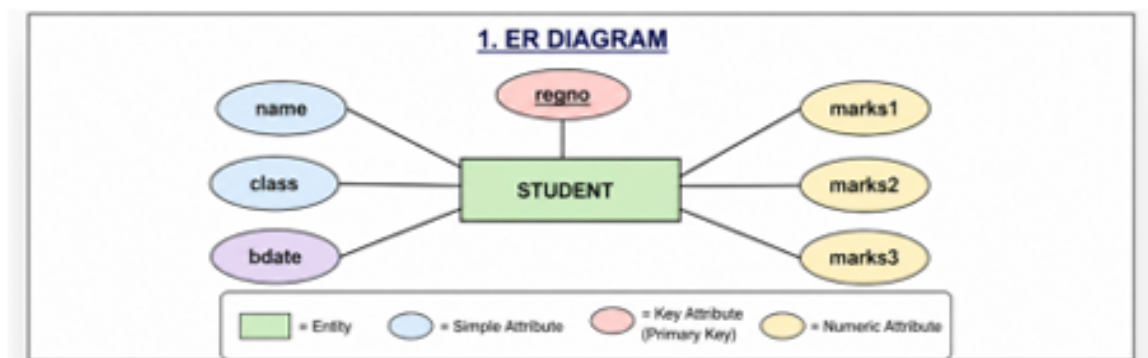


TABLE SCHEMA

Column Name		Data Type	Constraint
Name		Varchar(50)	NOT NULL
<u>Regno</u>		Varchar(15)	Primary key
Class		Varchar(20)	NOT NULL
Bdate		Date	
Marks1		Int	
Marks2		Int	
Marks3	Int		

1,b)

B) Consider the following database that maintain information about employees & Departments.

Employee (empid: int, ename: string, age:int, salary:int, #deptno:int)

Department (deptno:int, dname: string, #manager-id: int)

- i. Create the above tables by properly specifying the primary keys & foreign keys.
- ii. Enter at least 5 tuples for each relation.
- iii. Display emp-id & emp name whose salary lies between 10,000 and 50,000.
- iv. List empname & salary for all the employee working for CS Dept.
- v. Display empname & deptname for all the managers.

-- CREATE TABLES

-- Department Table

```
CREATE TABLE Department (  
    deptno INT PRIMARY KEY,  
    dname VARCHAR(50),  
    manager_id INT  
);
```

-- Employee Table

```
CREATE TABLE Employee (  
    empid INT PRIMARY KEY,  
    ename VARCHAR(50),  
    age INT,  
    salary INT,  
    deptno INT,  
    FOREIGN KEY (deptno) REFERENCES Department(deptno)  
);
```

-- INSERT AT LEAST 5 TUPLES

-- Insert into Department Table

```
INSERT INTO Department VALUES  
(1, 'HR', 101),  
(2, 'CS', 102),  
(3, 'Finance', 103),  
(4, 'Marketing', 104),  
(5, 'IT', 105);
```

-- Insert into Employee Table

```
INSERT INTO Employee VALUES  
(101, 'Amit', 25, 60000, 2),  
(102, 'Neha', 40, 50000, 1),  
(103, 'Raj', 45, 45000, 3),  
(104, 'Sara', 30, 15000, 4),  
(105, 'Priya', 28, 20000, 5),  
(106, 'Kiran', 24, 3000, 2),  
(107, 'Vijay', 32, 2000, 1);
```

*-- 3. Display empid & ename whose salary
-- is between 10,000 and 50,000*

```
SELECT empid, ename  
FROM Employee  
WHERE salary BETWEEN 10000 AND 50000;
```

*-- 4. List empname & salary for employees
-- working in CS Department*

```
SELECT e.ename, e.salary  
FROM Employee e  
JOIN Department d  
ON e.deptno = d.deptno  
WHERE d.dname = 'CS';
```

*-- 5. Display empname & deptname
-- for all managers*

```

SELECT e.ename, d.dname
FROM Employee e
JOIN Department d
ON e.empid = d.manager_id;

```

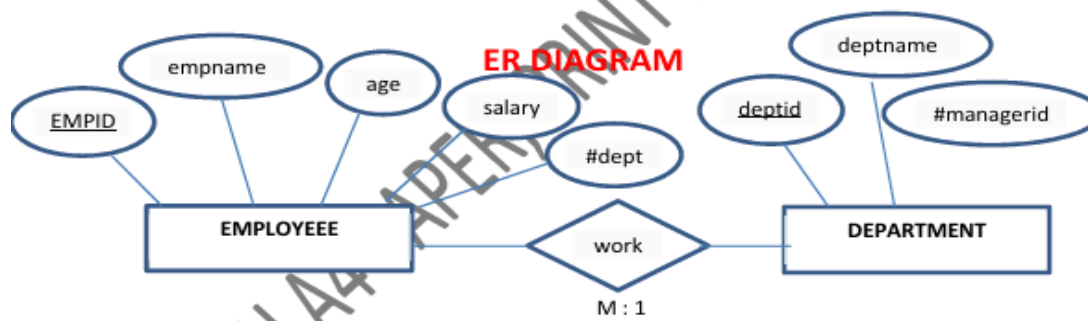


TABLE SCHEMA

EMPLOYEE

column name	Data type	constraint
Empname	Varchar(50)	Not null
Empid	Int	PK
Age	Int	
Salary	Decimal(10,2)	
Dept	int	

Department

column name	Data type	constraint
Deptid	Int	
Deptname	Varchar(50)	
Managerid	Int	FK